specify the problem, that forces us to state a problem in such a way to eliminate shared state[9], and handles the little synchronization still needed under the hood.

13.3 MapReduce k-means

To efficiently parallelize any algorithm, we first need to answer one question: how datapoints influence computation and what data do we really need at any time to perform a certain step?

In the case of both k-means and canopy clustering, the way we compute the various steps dictates a MapReduce implementation; let's examine each step of k-means separately[10]:

- In the *classification step*, when we assign points to canopies/clusters, for each point the operation is computed independently, and the only thing that matters is the list of centroids. So, we can shard the data however we like, as long as we pass all the centroids to each mapper;
- In the *re-centering* step, to update k-means' centroids, for each centroid we only need the data assigned to it, and each point is assigned to a single centroid, so we can partition the dataset and process each group separately;
- *Initialization* is trickier for k-means: we would need to draw points randomly from the full dataset, and this would seem to hinder parallelization; we can, however,